FDE PROGRAM | OPENAI CODEX

OpenAI Codex: Setup and Do's and Don'ts

A two-page card for Day 3 and every day after.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

What OpenAI Codex is

OpenAI Codex is an AI coding agent. In VS Code it can explain code, plan a change, edit files, run commands and tests, and show you every diff for review. You stay responsible for every line you accept.

Set up in five steps

- Install the OpenAI Codex extension (published by OpenAI) in VS Code.
- Open the Codex panel and sign in with the program's OpenAI API key. Paste the key only into the sign-in prompt.
- Choose the approved default model (docs/MODEL_GUIDE.md).
- Set the sandbox to workspace-write with approvals; use read-only to explore.
- Add an AGENTS.md to your repository from shared/templates/AGENTS_learner_template.md.

The structured workflow

Step	You	Codex
Describe	Goal, files involved, how to check it works	Asks questions if unclear
Plan	Ask for a plan; approve or edit it	Proposes small steps
Small steps	Approve one step at a time	Edits a few files per step
Test	Run tests after every step	Writes tests first if asked
Review	Read every diff; accept only what you can explain	Explains any line

Which model for what

Task	Model	Effort
Explain code, small edits, docstrings	Light model (for example GPT-6 Luna, if approved)	Low
Most feature work	Default: GPT-6.1 Sol	Medium
Hard debugging, large migrations	GPT-6.1 Sol high effort; GPT-6 Astra with trainer approval	High
Security review and patches	Default, with human verification	Medium

Task	Model	Effort
Retired models such as GPT-5.5	Do not use	n/a

Names checked 21 Oct 2026. Confirm against the approved model list.

Do

- Plan first for multi-step work; approve the plan, then let Codex build in small steps.
- Keep tasks small and include only the relevant files to save tokens.
- Write or generate tests first; run them after every change.
- Read every diff; accept only code you can explain.
- Keep AGENTS.md updated with build, test and style rules.
- Use workspace-write sandbox with approvals; read-only for exploration.
- Run bandit, pip-audit and npm audit on AI-generated code; log findings.
- Check your daily token usage; stop runaway tasks.

Don't

- Paste API keys, customer data or credentials into prompts or AGENTS.md.
- Use full-access or bypass modes on any laptop with real credentials.
- Ask open-ended 'improve everything' requests or paste the whole codebase.
- Use a top-tier model or high reasoning effort for routine edits.
- Merge AI code without tests and a human review.
- Use models outside the program's approved list.

Vibe coding vs structured coding: accepting whatever the tool writes is fine for a throwaway prototype. With customers, always use a goal, a plan, small steps, tests and a human review.